

Módulo 17 - Manipulação de Dados: UPDATE e DELETE

Objetivos de Aprendizagem

Ao final deste módulo, o aluno será capaz de:

- Compreender e utilizar o comando UPDATE para modificar dados existentes
- Aplicar diferentes estratégias de atualização de dados
- Utilizar UPDATE com subconsultas e JOINS
- Compreender e utilizar o comando DELETE para remover dados
- Aplicar diferentes estratégias de exclusão de dados
- Implementar DELETE com condições e subconsultas
- Entender os riscos e precauções ao modificar/deletar dados
- Aplicar boas práticas de segurança em operações destrutivas
- Trabalhar com transações para garantir integridade dos dados
- Compreender cascata de exclusões e integridade referencial

Conteúdo Teórico

PARTE 1: COMANDO UPDATE - MODIFICANDO DADOS EXISTENTES

O comando UPDATE é uma operação fundamental de DML (Data Manipulation Language) que permite modificar valores de colunas em registros existentes. É uma das operações mais utilizadas no dia a dia, mas também uma das mais perigosas se não for utilizada com cuidado.

1.1 Conceitos Fundamentais do UPDATE

O que é o UPDATE?

- Comando SQL para modificar dados existentes em tabelas
- Permite alterar uma ou múltiplas colunas simultaneamente
- Pode afetar uma ou múltiplas linhas em uma única operação
- É uma operação destrutiva - modifica permanentemente os dados após COMMIT

Por que o UPDATE é importante?

- Dados mudam ao longo do tempo e precisam ser atualizados
- Correção de erros e inconsistências em dados existentes
- Atualização de status e estados em sistemas (ex: pedido pendente → entregue)
- Manutenção da qualidade e precisão dos dados
- Implementação de regras de negócio dinâmicas

Características principais:

- **Seletivo:** Pode atualizar registros específicos usando WHERE
- **Flexível:** Pode atualizar uma ou múltiplas colunas

- **Condicional:** Suporta subconsultas e expressões complexas
- **Transacional:** Pode ser revertido com ROLLBACK antes do COMMIT
- **Perigoso:** Sem WHERE, atualiza TODOS os registros da tabela

1.2 Sintaxe Básica do UPDATE

```
UPDATE nome_tabela
SET coluna1 = valor1,
    coluna2 = valor2,
    ...
WHERE condição;
```

⚠ **AVISO CRÍTICO:** SEMPRE use WHERE no UPDATE, exceto se realmente deseja atualizar TODAS as linhas!

1.3 UPDATE Simples - Atualizando uma Coluna

Exemplo 1: Atualizar nome de um usuário específico

```
-- Atualizar nome de um único usuário
UPDATE usuario
SET nome_usuario = 'Carlos Silva Júnior'
WHERE id_usuario = 1;
-- Resultado: 1 linha atualizada
```

Exemplo 2: Atualizar senha de um usuário

```
-- Atualizar senha de usuário por email
UPDATE usuario
SET senha = 'nova_senha_segura'
WHERE email = 'carlos@email.com';
```

Exemplo 3: Marcar usuário como inativo

```
-- Desativar conta de usuário
UPDATE usuario
SET ativo = 'N'
WHERE id_usuario = 6;
```

Exemplo 4: Atualizar data de último acesso

```
-- Registrar acesso do usuário
UPDATE usuario
SET ultimo_acesso = SYSTIMESTAMP
WHERE id_usuario = 1;
```

Exemplo 5: Corrigir país de origem de artista

```
-- Corrigir informação incorreta
UPDATE artista
SET pais_origem = 'Brasil'
WHERE id_artista = 2;
```

1.4 UPDATE com Múltiplas Colunas

Frequentemente precisamos atualizar várias colunas ao mesmo tempo para manter a consistência dos dados.

Exemplo 6: Atualizar múltiplos campos de usuário

```
-- Atualizar perfil completo do usuário
UPDATE usuario
SET nome_usuario = 'João Pedro Silva',
    email = 'joao.pedro@email.com',
    pais = 'Brasil',
    ativo = 'S',
    ultimo_acesso = SYSTIMESTAMP
WHERE id_usuario = 3;
```

Exemplo 7: Atualizar informações do artista

```
-- Atualizar dados completos do artista
UPDATE artista
SET nome_artista = 'The Beatles Remastered',
    website = 'https://www.beatles.com',
    biografia = 'Banda britânica de rock formada em Liverpool em 1960. Considerada uma das bandas mais influentes de todos os tempos.',
    ativo = 'S'
WHERE id_artista = 1;
```

Exemplo 8: Atualizar dados de álbum

```
-- Corrigir informações de álbum
UPDATE album
```

```
SET titulo = 'Rock Forever – Deluxe Edition',
    numero_faixas = 12,
    duracao_total = 4200,
    tipo_album = 'ALBUM'
WHERE id_album = 1;
```

Exemplo 9: Atualizar playlist

```
-- Atualizar metadados da playlist
UPDATE playlist
SET nome_playlist = 'Favoritas Rock 2024',
    descricao = 'As melhores músicas de rock do ano',
    publica = 'S',
    data_atualizacao = SYSTIMESTAMP
WHERE id_playlist = 1;
```

Exemplo 10: Atualizar tipo de assinatura

```
-- Ajustar plano de assinatura
UPDATE tipo_assinatura
SET preco_mensal = 34.90,
    qualidade_audio = 'Ultra Alta',
    descricao = 'Plano premium com qualidade superior'
WHERE id_tipo_assinatura = 2;
```

1.5 UPDATE com Expressões e Cálculos

É possível usar expressões matemáticas e funções SQL nos valores do UPDATE.

Exemplo 11: Incrementar contador de reproduções

```
-- Adicionar mais reproduções a uma música
UPDATE musica
SET total_reproducoes = total_reproducoes + 1
WHERE id_musica = 1;
```

Exemplo 12: Aumentar preço em percentual

```
-- Aplicar reajuste de 10% em todos os planos
UPDATE tipo_assinatura
SET preco_mensal = preco_mensal * 1.10
WHERE ativo = 'S';
```

Exemplo 13: Reduzir duração de músicas (teste)

```
-- Ajustar duração de músicas de teste
UPDATE musica
SET duracao = duracao - 10
WHERE id_album = 1 AND duracao > 300;
```

Exemplo 14: Calcular e atualizar idade do artista

```
-- Atualizar campo calculado (se existisse)
UPDATE artista
SET numero_membros = numero_membros + 1
WHERE id_artista = 1;
```

Exemplo 15: Atualizar com concatenação de strings

```
-- Adicionar prefixo ao nome
UPDATE genero
SET nome_genero = 'Gênero: ' || nome_genero
WHERE id_genero = 1;
```

1.6 UPDATE com Condições WHERE Complexas

Podemos usar múltiplas condições, operadores lógicos e comparações para selecionar registros específicos.

Exemplo 16: UPDATE com múltiplas condições AND

```
-- Atualizar apenas usuários específicos
UPDATE usuario
SET ativo = 'N',
    ultimo_acesso = NULL
WHERE pais = 'Brasil'
    AND data_cadastro < TO_DATE('2024-01-01', 'YYYY-MM-DD')
    AND ativo = 'S';
```

Exemplo 17: UPDATE com condição OR

```
-- Marcar músicas explícitas
UPDATE musica
SET explicita = 'S'
```

```
WHERE id_musica IN (1, 5, 10, 15)
OR titulo LIKE '%Explicit%';
```

Exemplo 18: UPDATE com condição NOT

```
-- Atualizar todos os artistas exceto um país
UPDATE artista
SET ativo = 'S'
WHERE pais_origem != 'Brasil'
AND data_inicio_carreira IS NOT NULL;
```

Exemplo 19: UPDATE com BETWEEN

```
-- Atualizar músicas dentro de um range de duração
UPDATE musica
SET explicita = 'N'
WHERE duracao BETWEEN 180 AND 300;
```

Exemplo 20: UPDATE com IS NULL / IS NOT NULL

```
-- Preencher biografia vazia com texto padrão
UPDATE artista
SET biografia = 'Biografia a ser atualizada'
WHERE biografia IS NULL;
```

1.7 UPDATE com Subconsultas (Subqueries)

Subconsultas permitem usar dados de outras tabelas ou consultas complexas como valores no UPDATE.

Exemplo 21: UPDATE usando valor de outra tabela

```
-- Atualizar gênero da música baseado no gênero do álbum
UPDATE musica
SET id_genero = (
    SELECT id_genero
    FROM album
    WHERE album.id_album = musica.id_album
)
WHERE id_genero IS NULL;
```

Exemplo 22: UPDATE baseado em agregação

```
-- Atualizar total de músicas em playlist
UPDATE playlist
SET total_musicas = (
    SELECT COUNT(*)
    FROM playlist_musica
    WHERE playlist_musica.id_playlist = playlist.id_playlist
);
```

Exemplo 23: UPDATE com subconsulta no WHERE

```
-- Ativar artistas que têm álbuns
UPDATE artista
SET ativo = 'S'
WHERE id_artista IN (
    SELECT DISTINCT id_artista
    FROM album
    WHERE data_lancamento > TO_DATE('2010-01-01', 'YYYY-MM-DD')
);
```

Exemplo 24: UPDATE com subconsulta correlacionada

```
-- Atualizar duração total do álbum baseado nas músicas
UPDATE album a
SET duracao_total = (
    SELECT SUM(duracao)
    FROM musica m
    WHERE m.id_album = a.id_album
)
WHERE EXISTS (
    SELECT 1
    FROM musica m
    WHERE m.id_album = a.id_album
);
```

Exemplo 25: UPDATE com MAX/MIN de subconsulta

```
-- Atualizar status do usuário mais recente
UPDATE usuario
SET ativo = 'S'
WHERE data_cadastro = (
    SELECT MAX(data_cadastro)
    FROM usuario
    WHERE pais = 'Brasil'
);
```

1.8 UPDATE com JOINs (Sintaxe específica do banco de dados)

Alguns bancos de dados suportam UPDATE com JOIN. Oracle usa sintaxe diferente (subconsultas).

Exemplo 26: UPDATE usando EXISTS (equivalente a JOIN)

```
-- Atualizar músicas de artistas brasileiros
UPDATE musica
SET explicita = 'N'
WHERE EXISTS (
    SELECT 1
    FROM album al
    JOIN artista ar ON al.id_artista = ar.id_artista
    WHERE al.id_album = musica.id_album
    AND ar.pais_origem = 'Brasil'
);
```

Exemplo 27: UPDATE com múltiplas subconsultas relacionadas

```
-- Atualizar preço da assinatura baseado no usuário
UPDATE assinatura
SET valor_pago = (
    SELECT preco_mensal
    FROM tipo_assinatura
    WHERE tipo_assinatura.id_tipo_assinatura =
assinatura.id_tipo_assinatura
)
WHERE data_inicio >= TO_DATE('2024-01-01', 'YYYY-MM-DD');
```

Exemplo 28: Sincronizar informações entre tabelas

```
-- Sincronizar total de músicas na playlist
UPDATE playlist p
SET total_musicas = (
    SELECT COUNT(*)
    FROM playlist_musica pm
    WHERE pm.id_playlist = p.id_playlist
),
duracao_total = (
    SELECT COALESCE(SUM(m.duracao), 0)
    FROM playlist_musica pm
    JOIN musica m ON pm.id_musica = m.id_musica
    WHERE pm.id_playlist = p.id_playlist
);
```

1.9 UPDATE em Massa (Bulk Update)

Atualizações que afetam múltiplos registros de uma só vez.

Exemplo 29: Ativar todos os usuários de um país

```
-- Ativar todos usuários brasileiros
UPDATE usuario
SET ativo = 'S'
WHERE pais = 'Brasil';
-- Pode afetar múltiplas linhas
```

Exemplo 30: Normalizar dados em lote

```
-- Padronizar formato de país
UPDATE usuario
SET pais = 'Estados Unidos'
WHERE pais IN ('USA', 'EUA', 'United States');
```

Exemplo 31: Aplicar desconto em massa

```
-- Aplicar desconto em todos planos ativos
UPDATE tipoassinatura
SET preco_mensal = preco_mensal * 0.85
WHERE ativo = 'S'
AND nome_plano != 'Free';
```

Exemplo 32: Resetar contadores

```
-- Resetar reproduções de músicas antigas
UPDATE musica
SET total_reproducoes = 0
WHERE data_upload < TO_DATE('2020-01-01', 'YYYY-MM-DD');
```

Exemplo 33: Atualização global (cuidado!)

```
-- Marcar todas as músicas como não explícitas (exemplo de uso com
WHERE)
UPDATE musica
SET explicita = 'N'
WHERE explicita = 'S' OR explicita IS NULL;
```

1.10 UPDATE com CASE - Lógica Condicional

O CASE permite aplicar diferentes valores baseado em condições.

Exemplo 34: UPDATE com CASE simples

```
-- Atualizar status baseado em data
UPDATE assinatura
SET status_assinatura = CASE
    WHEN data_fim IS NULL THEN 'ATIVA'
    WHEN data_fim < SYSDATE THEN 'EXPIRADA'
    WHEN data_fim >= SYSDATE THEN 'ATIVA'
    ELSE 'DESCONHECIDA'
END;
```

Exemplo 35: Ajustar preços por categoria

```
-- Aplicar diferentes descontos por tipo de plano
UPDATE tipo_assinatura
SET preco_mensal = CASE
    WHEN nome_plano = 'Premium' THEN preco_mensal * 0.9
    WHEN nome_plano = 'Família' THEN preco_mensal * 0.85
    WHEN nome_plano = 'Estudante' THEN preco_mensal * 0.95
    ELSE preco_mensal
END
WHERE ativo = 'S';
```

Exemplo 36: Classificar músicas por duração

```
-- Adicionar tag baseado em duração (se houvesse coluna tag)
UPDATE musica
SET explicita = CASE
    WHEN duracao < 180 THEN 'N'
    WHEN duracao > 600 THEN 'S'
    ELSE explicita
END;
```

Exemplo 37: UPDATE condicional em múltiplas colunas

```
-- Ajustar dados de artista baseado em número de membros
UPDATE artista
SET ativo = CASE
    WHEN numero_membros > 10 THEN 'S'
    WHEN numero_membros IS NULL THEN 'N'
    ELSE ativo
END,
pais_origem = CASE
```

```
    WHEN pais_origem IS NULL THEN 'Desconhecido'
    ELSE pais_origem
END;
```

Exemplo 38: Normalização condicional

```
-- Normalizar valores de país
UPDATE usuario
SET pais = CASE
    WHEN pais IN ('USA', 'United States', 'US') THEN 'Estados Unidos'
    WHEN pais IN ('UK', 'England', 'United Kingdom') THEN 'Reino Unido'
    WHEN pais IN ('PT', 'Port') THEN 'Portugal'
    ELSE pais
END;
```

1.11 Boas Práticas e Segurança no UPDATE

Regras de Ouro para UPDATE Seguro:

1. SEMPRE teste com SELECT primeiro

```
-- PASSO 1: Verificar quais registros serão afetados
SELECT *
FROM usuario
WHERE id_usuario = 1;

-- PASSO 2: Executar o UPDATE
UPDATE usuario
SET nome_usuario = 'Novo Nome'
WHERE id_usuario = 1;
```

2. Use transações explícitas

```
-- Iniciar transação implicitamente (primeiro DML)
UPDATE usuario
SET ativo = 'N'
WHERE id_usuario = 1;

-- Verificar resultado
SELECT * FROM usuario WHERE id_usuario = 1;

-- Se correto: confirmar
COMMIT;

-- Se errado: desfazer
-- ROLLBACK;
```

3. Sempre use WHERE (exceto em casos muito específicos)

```
-- ❌ PERIGO: Atualiza TODAS as linhas
UPDATE usuario SET ativo = 'N';

-- ✅ CORRETO: Atualiza apenas linha específica
UPDATE usuario SET ativo = 'N' WHERE id_usuario = 1;
```

4. Faça backup antes de UPDATE em massa

```
-- Criar backup da tabela
CREATE TABLE usuario_backup AS
SELECT * FROM usuario;

-- Executar UPDATE
UPDATE usuario
SET ativo = 'N'
WHERE data_cadastro < TO_DATE('2020-01-01', 'YYYY-MM-DD');
```

5. Valide restrições e integridade

```
-- Verificar constraints antes de atualizar
SELECT constraint_name, constraint_type
FROM user_constraints
WHERE table_name = 'USUARIO';

-- UPDATE respeitando constraints
UPDATE usuario
SET email = 'novo@email.com' -- Deve ser único e válido
WHERE id_usuario = 1;
```

Exemplo 39: Padrão de UPDATE seguro completo

```
-- PASSO 1: Consultar dados atuais
SELECT id_usuario, nome_usuario, email, ativo
FROM usuario
WHERE id_usuario = 5;

-- PASSO 2: Executar UPDATE
UPDATE usuario
SET nome_usuario = 'Pedro Costa Silva',
    email = 'pedro.costa@email.com',
    ativo = 'S'
```

```
WHERE id_usuario = 5;

-- PASSO 3: Verificar mudanças
SELECT id_usuario, nome_usuario, email, ativo
FROM usuario
WHERE id_usuario = 5;

-- PASSO 4: Confirmar ou reverter
-- COMMIT; -- Se estiver correto
-- ROLLBACK; -- Se estiver errado
```

Exemplo 40: UPDATE com validação prévia

```
-- Verificar quantos registros serão afetados
SELECT COUNT(*) as total_afetado
FROM musica
WHERE id_album = 1;

-- Executar UPDATE apenas se quantidade for esperada
UPDATE musica
SET id_genero = 1
WHERE id_album = 1
    AND (SELECT COUNT(*) FROM musica WHERE id_album = 1) <= 10;
```

PARTE 2: COMANDO DELETE - REMOVENDO DADOS

O comando DELETE é usado para remover registros de uma tabela. É uma das operações mais perigosas em SQL, pois uma vez confirmada (COMMIT), os dados são permanentemente removidos (exceto se houver backups).

2.1 Conceitos Fundamentais do DELETE

O que é o DELETE?

- Comando SQL para remover registros de tabelas
- Remove linhas completas (não colunas individuais)
- Pode remover um ou múltiplos registros em uma operação
- É uma operação destrutiva - dados são perdidos após COMMIT
- Mantém a estrutura da tabela (diferente de DROP TABLE)

Por que o DELETE é importante?

- Remoção de dados obsoletos ou incorretos
- Manutenção de políticas de retenção de dados (LGPD, GDPR)
- Limpeza de dados de teste
- Implementação de regras de negócio (ex: cancelamento de conta)
- Gerenciamento de espaço e performance do banco

Características principais:

- **Seletivo:** Usa WHERE para escolher registros específicos
- **Completo:** Remove a linha inteira, não colunas individuais
- **Transacional:** Pode ser revertido com ROLLBACK antes do COMMIT
- **Cascata:** Pode afetar tabelas relacionadas (ON DELETE CASCADE)
- **Extremamente perigoso:** Sem WHERE, deleta TODOS os registros!

2.2 Sintaxe Básica do DELETE

```
DELETE FROM nome_tabela  
WHERE condição;
```

⚠ **AVISO CRÍTICO:** SEMPRE use WHERE no DELETE, exceto se realmente deseja deletar TODAS as linhas!

2.3 DELETE Simples - Removendo Registros Específicos

Exemplo 41: Deletar um usuário específico

```
-- Deletar usuário por ID  
DELETE FROM usuario  
WHERE id_usuario = 10;  
-- Resultado: 1 linha removida
```

Exemplo 42: Deletar por email

```
-- Remover usuário por email  
DELETE FROM usuario  
WHERE email = 'usuario.teste@email.com';
```

Exemplo 43: Deletar playlist específica

```
-- Remover playlist  
DELETE FROM playlist  
WHERE id_playlist = 8;
```

Exemplo 44: Deletar música

```
-- Remover música específica  
DELETE FROM musica  
WHERE id_musica = 35;
```

Exemplo 45: Deletar gênero não utilizado

```
-- Remover gênero que não tem músicas associadas
DELETE FROM genero
WHERE id_genero = 10
  AND NOT EXISTS (
    SELECT 1 FROM musica WHERE id_genero = 10
  );
```

2.4 DELETE com Condições WHERE Complexas

Exemplo 46: DELETE com múltiplas condições AND

```
-- Deletar usuários inativos de determinado país
DELETE FROM usuario
WHERE ativo = 'N'
  AND pais = 'Brasil'
  AND data_cadastro < TO_DATE('2020-01-01', 'YYYY-MM-DD');
```

Exemplo 47: DELETE com condição OR

```
-- Deletar músicas curtas ou antigas
DELETE FROM musica
WHERE duracao < 60
  OR data_upload < TO_DATE('2015-01-01', 'YYYY-MM-DD');
```

Exemplo 48: DELETE com IN

```
-- Deletar múltiplos usuários específicos
DELETE FROM usuario
WHERE id_usuario IN (11, 12, 13, 14, 15);
```

Exemplo 49: DELETE com LIKE

```
-- Deletar playlists de teste
DELETE FROM playlist
WHERE nome_playlist LIKE '%teste%'
  OR nome_playlist LIKE '%test%';
```

Exemplo 50: DELETE com BETWEEN

```
-- Deletar músicas dentro de um range de IDs
DELETE FROM musica
WHERE id_musica BETWEEN 100 AND 200
AND total_reproducoes = 0;
```

2.5 DELETE com Subconsultas

Exemplo 51: DELETE usando subconsulta no WHERE

```
-- Deletar músicas de artistas inativos
DELETE FROM musica
WHERE id_album IN (
    SELECT a.id_album
    FROM album a
    JOIN artista ar ON a.id_artista = ar.id_artista
    WHERE ar.ativo = 'N'
);
```

Exemplo 52: DELETE baseado em agregação

```
-- Deletar playlists vazias
DELETE FROM playlist
WHERE id_playlist NOT IN (
    SELECT DISTINCT id_playlist
    FROM playlist_musica
);
```

Exemplo 53: DELETE com EXISTS

```
-- Deletar usuários sem assinaturas
DELETE FROM usuario
WHERE NOT EXISTS (
    SELECT 1
    FROM assinatura
    WHERE assinatura.id_usuario = usuario.id_usuario
);
```

Exemplo 54: DELETE correlacionado complexo


```
-- Deletar histórico antigo de usuários inativos
DELETE FROM historico_reproducao
WHERE id_usuario IN (
    SELECT id_usuario
    FROM usuario
    WHERE ativo = 'N'
)
AND data_reproducao < SYSTIMESTAMP - INTERVAL '1' YEAR;
```

Exemplo 55: DELETE com subconsulta de agregação

```
-- Deletar artistas sem álbuns
DELETE FROM artista
WHERE id_artista NOT IN (
    SELECT DISTINCT id_artista
    FROM album
);
```

2.6 DELETE em Massa (Bulk Delete)

Exemplo 56: Limpar dados de teste

```
-- Deletar todos os registros de teste
DELETE FROM usuario
WHERE email LIKE '%@test.com'
    OR nome_usuario LIKE '%Test%';
```

Exemplo 57: Limpeza de histórico antigo

```
-- Deletar reproduções antigas
DELETE FROM historico_reproducao
WHERE data_reproducao < ADD_MONTHS(SYSDATE, -12);
-- Remove histórico com mais de 1 ano
```

Exemplo 58: Remover músicas não publicadas

```
-- Deletar músicas sem álbum ou dados incompletos
DELETE FROM musica
WHERE arquivo_url IS NULL
    OR duracao <= 0
    OR titulo IS NULL;
```

Exemplo 59: Limpeza de playlists inativas

```
-- Deletar playlists não atualizadas há muito tempo
DELETE FROM playlist
WHERE data_atualizacao < ADD_MONTHS(SYSDATE, -24)
AND total_musicas = 0;
```

Exemplo 60: Remover assinaturas expiradas

```
-- Deletar assinaturas canceladas antigas
DELETE FROM assinatura
WHERE status_assinatura = 'CANCELADA'
AND data_fim < ADD_MONTHS(SYSDATE, -6);
```

2.7 DELETE com Integridade Referencial

Entender como DELETE interage com chaves estrangeiras é crucial.

Conceitos importantes:

- **ON DELETE CASCADE:** Remove registros dependentes automaticamente
- **ON DELETE SET NULL:** Define FK como NULL em registros dependentes
- **ON DELETE RESTRICT:** Impede DELETE se houver dependentes
- **ON DELETE NO ACTION:** Similar ao RESTRICT

Exemplo 61: DELETE com CASCADE implícito

```
-- Ao deletar um álbum, as músicas também são deletadas (se CASCADE)
DELETE FROM album
WHERE id_album = 10;
-- Se há FOREIGN KEY com ON DELETE CASCADE, músicas são removidas também
```

Exemplo 62: Verificar dependências antes de DELETE

```
-- Verificar se artista tem álbuns antes de deletar
SELECT COUNT(*) as total_albums
FROM album
WHERE id_artista = 10;

-- Se total_albums = 0, seguro deletar
DELETE FROM artista
WHERE id_artista = 10
AND NOT EXISTS (SELECT 1 FROM album WHERE id_artista = 10);
```

Exemplo 63: DELETE de tabela relacionamento (N:M)

```
-- Remover músicas de uma playlist
DELETE FROM playlist_musica
WHERE id_playlist = 5
    AND id_musica IN (1, 2, 3);
```

Exemplo 64: DELETE preservando integridade

```
-- Deletar usuário e seus relacionamentos manualmente
-- Passo 1: Deletar histórico de reprodução
DELETE FROM historico_reproducao WHERE id_usuario = 15;

-- Passo 2: Deletar assinaturas
DELETE FROM assinatura WHERE id_usuario = 15;

-- Passo 3: Deletar playlists e seus relacionamentos
DELETE FROM playlist_musica
WHERE id_playlist IN (SELECT id_playlist FROM playlist WHERE id_usuario
= 15);

DELETE FROM playlist WHERE id_usuario = 15;

-- Passo 4: Finalmente deletar usuário
DELETE FROM usuario WHERE id_usuario = 15;

-- Confirmar todas as operações
COMMIT;
```

Exemplo 65: DELETE em ordem correta (dependências)

```
-- Ordem: mais dependente para menos dependente
-- 1. Histórico
DELETE FROM historico_reproducao WHERE id_musica = 20;

-- 2. Playlist_musica
DELETE FROM playlist_musica WHERE id_musica = 20;

-- 3. Música
DELETE FROM musica WHERE id_musica = 20;
```

2.8 DELETE vs TRUNCATE

Diferenças importantes:

DELETE:

- Remove linhas individualmente
- Pode usar WHERE (seletivo)
- Gera logs de transação (pode ser lento)
- Pode ser revertido com ROLLBACK
- Dispara triggers
- Mantém espaço alocado

TRUNCATE:

- Remove todas as linhas de uma vez
- Não usa WHERE (remove tudo)
- Mínimo de logs (muito rápido)
- Não pode ser revertido (geralmente)
- Não dispara triggers
- Libera espaço

Exemplo 66: DELETE seletivo vs TRUNCATE completo

```
-- DELETE seletivo (removível antes do COMMIT)
DELETE FROM historico_reproducao
WHERE data_reproducao < TO_DATE('2023-01-01', 'YYYY-MM-DD');
-- Pode fazer ROLLBACK

-- TRUNCATE completo (cuidado!)
TRUNCATE TABLE historico_reproducao;
-- Remove TUDO, muito rápido, não pode fazer ROLLBACK (em muitos DBs)
```

2.9 Padrões de DELETE Seguro

Exemplo 67: Padrão de DELETE seguro completo

```
-- PASSO 1: Verificar o que será deletado
SELECT *
FROM usuario
WHERE id_usuario = 20;

-- PASSO 2: Contar quantos registros
SELECT COUNT(*)
FROM usuario
WHERE id_usuario = 20;

-- PASSO 3: Verificar dependências
SELECT 'playlists' as tabela, COUNT(*) as total
FROM playlist WHERE id_usuario = 20
UNION ALL
SELECT 'assinaturas', COUNT(*)
FROM assinatura WHERE id_usuario = 20
UNION ALL
```

```

SELECT 'historico', COUNT(*)
FROM historico_reproducao WHERE id_usuario = 20;

-- PASSO 4: Executar DELETE
DELETE FROM usuario
WHERE id_usuario = 20;

-- PASSO 5: Verificar resultado
SELECT COUNT(*) as registros_restantes
FROM usuario
WHERE id_usuario = 20;
-- Deve retornar 0

-- PASSO 6: Confirmar ou reverter
-- COMMIT; -- Se correto
-- ROLLBACK; -- Se errado

```

Exemplo 68: DELETE com backup antes

```

-- Criar backup antes de DELETE em massa
CREATE TABLE musica_backup AS
SELECT *
FROM musica
WHERE id_album IN (5, 6, 7);

-- Executar DELETE
DELETE FROM musica
WHERE id_album IN (5, 6, 7);

-- Verificar
SELECT COUNT(*) FROM musica WHERE id_album IN (5, 6, 7);

-- Se correto: COMMIT e depois dropar backup
-- COMMIT;
-- DROP TABLE musica_backup;

-- Se errado: ROLLBACK
-- ROLLBACK;

```

Exemplo 69: DELETE incremental para grandes volumes

```

-- Deletar em lotes para evitar locks longos
DELETE FROM historico_reproducao
WHERE data_reproducao < TO_DATE('2022-01-01', 'YYYY-MM-DD')
  AND ROWNUM <= 1000;
-- Executa múltiplas vezes até não haver mais linhas

COMMIT;

```

```
-- Repete até COUNT retornar 0
SELECT COUNT(*)
FROM historico_reproducao
WHERE data_reproducao < TO_DATE('2022-01-01', 'YYYY-MM-DD');
```

Exemplo 70: DELETE com logging manual

```
-- Registrar DELETE em tabela de auditoria (se existisse)
-- Antes do DELETE, inserir em log
INSERT INTO auditoria_deletes (tabela, id_registro, usuario,
data_operacao)
SELECT 'usuario', id_usuario, USER, SYSDATE
FROM usuario
WHERE id_usuario = 25;

-- Executar DELETE
DELETE FROM usuario
WHERE id_usuario = 25;

COMMIT;
```

2.10 DELETE - Casos de Uso Práticos

Exemplo 71: Implementar LGPD - Direito ao Esquecimento

```
-- Remover todos os dados de um usuário (LGPD)
-- Primeiro criar auditoria
INSERT INTO auditoria_lgpd (id_usuario, data_solicitacao, data_execucao)
VALUES (30, SYSDATE, SYSDATE);

-- Remover dados pessoais em cascata
DELETE FROM historico_reproducao WHERE id_usuario = 30;
DELETE FROM assinatura WHERE id_usuario = 30;
DELETE FROM playlist_musica
WHERE id_playlist IN (SELECT id_playlist FROM playlist WHERE id_usuario
= 30);
DELETE FROM playlist WHERE id_usuario = 30;
DELETE FROM usuario WHERE id_usuario = 30;

COMMIT;
```

Exemplo 72: Limpeza de dados duplicados

```
-- Deletar usuários duplicados (mantendo o mais recente)
DELETE FROM usuario
```

```

WHERE id_usuario IN (
    SELECT u1.id_usuario
    FROM usuario u1
    JOIN usuario u2 ON u1.email = u2.email
                   AND u1.id_usuario < u2.id_usuario
);

```

Exemplo 73: Remover dados de teste do ambiente de produção

```

-- Identificar e remover dados de teste
DELETE FROM usuario
WHERE email LIKE '%@teste.com'
      OR email LIKE '%@test.com'
      OR nome_usuario LIKE 'Test%'
      OR nome_usuario LIKE 'Teste%';

DELETE FROM artista
WHERE nome_artista LIKE '%Test%'
      OR nome_artista LIKE '%Teste%';

COMMIT;

```

Exemplo 74: Limpeza por política de retenção

```

-- Política: Manter histórico apenas dos últimos 2 anos
DELETE FROM historico_reproducao
WHERE data_reproducao < ADD_MONTHS(SYSDATE, -24);

-- Política: Remover assinaturas canceladas há mais de 1 ano
DELETE FROM assinatura
WHERE status_assinatura = 'CANCELADA'
      AND data_fim < ADD_MONTHS(SYSDATE, -12);

COMMIT;

```

Exemplo 75: Limpeza de dados órfãos

```

-- Remover registros que perderam integridade referencial
-- Playlists sem usuário (se FK permitir NULL)
DELETE FROM playlist
WHERE id_usuario NOT IN (SELECT id_usuario FROM usuario);

-- Músicas sem álbum válido
DELETE FROM musica
WHERE id_album NOT IN (SELECT id_album FROM album);

```

2.11 Boas Práticas e Segurança no DELETE

Regras de Ouro para DELETE Seguro:

1. SEMPRE teste com SELECT COUNT primeiro

```
-- NUNCA execute direto
DELETE FROM usuario WHERE pais = 'Brasil';

-- SEMPRE faça assim:
-- Passo 1: Visualizar
SELECT * FROM usuario WHERE pais = 'Brasil';

-- Passo 2: Contar
SELECT COUNT(*) FROM usuario WHERE pais = 'Brasil';

-- Passo 3: Se quantidade esperada, executar
DELETE FROM usuario WHERE pais = 'Brasil';

-- Passo 4: Verificar
SELECT COUNT(*) FROM usuario WHERE pais = 'Brasil';

-- Passo 5: COMMIT ou ROLLBACK
```

2. Use transações explícitas

```
-- Sempre trabalhe com consciência de transação
DELETE FROM musica WHERE id_musica = 50;

-- Verificar se deletou o esperado
SELECT * FROM musica WHERE id_musica = 50;
-- Deve retornar 0 linhas

-- Confirmar ou reverter
COMMIT; -- Se correto
-- ROLLBACK; -- Se errado
```

3. Nunca delete sem WHERE (exceto se realmente quiser tudo)

```
-- ❌ EXTREMO PERIGO: Deleta TUDO
DELETE FROM usuario;

-- ✅ CORRETO: Específico
DELETE FROM usuario WHERE id_usuario = 1;

-- Se realmente precisa deletar tudo, seja explícito
```



```
DELETE FROM usuario WHERE 1=1; -- Deixa claro que é intencional
-- Ou melhor, use TRUNCATE se for tudo mesmo
```

4. Faça backup antes de DELETE em massa

```
-- Backup antes de operação perigosa
CREATE TABLE usuario_backup AS
SELECT * FROM usuario;

-- Executar DELETE
DELETE FROM usuario
WHERE data_cadastro < TO_DATE('2020-01-01', 'YYYY-MM-DD');

-- Se deu errado, restaurar do backup
-- INSERT INTO usuario SELECT * FROM usuario_backup WHERE ...;
```

5. Verifique dependências antes de deletar

```
-- Verificar o que será afetado
SELECT
    'Playlists' as item,
    COUNT(*) as quantidade
FROM playlist
WHERE id_usuario = 10
UNION ALL
SELECT
    'Assinaturas',
    COUNT(*)
FROM assinatura
WHERE id_usuario = 10;

-- Só deletar depois de verificar impacto
```

6. Documente DELETES importantes

```
-- Documentar no código o motivo do DELETE
-- MOTIVO: Limpeza de dados de teste do ambiente de produção
-- DATA: 2024-11-12
-- RESPONSÁVEL: Admin
DELETE FROM usuario
WHERE email LIKE '%@test.com';

COMMIT;
```

7. Use soft delete quando apropriado

```

-- Em vez de DELETE físico, marcar como inativo (soft delete)
-- Melhor prática em muitos casos
UPDATE usuario
SET ativo = 'N',
    data_inativacao = SYSDATE
WHERE id_usuario = 10;

-- Em vez de:
-- DELETE FROM usuario WHERE id_usuario = 10;

```

2.12 Troubleshooting - Problemas Comuns

Problema 1: DELETE bloqueado por constraint

```

-- Erro: Cannot delete because of foreign key constraint

-- Solução 1: Deletar dependentes primeiro
DELETE FROM playlist_musica WHERE id_playlist = 5;
DELETE FROM playlist WHERE id_playlist = 5;

-- Solução 2: Se FK tem CASCADE, deletar pai apenas
DELETE FROM playlist WHERE id_playlist = 5;
-- Dependentes são removidos automaticamente

```

Problema 2: DELETE muito lento

```

-- Problema: DELETE de milhões de linhas trava

-- Solução: Deletar em lotes
DECLARE
    v_deleted NUMBER;
BEGIN
    LOOP
        DELETE FROM historico_reproducao
        WHERE data_reproducao < TO_DATE('2020-01-01', 'YYYY-MM-DD')
          AND ROWNUM <= 10000;

        v_deleted := SQL%ROWCOUNT;
        COMMIT;

        EXIT WHEN v_deleted = 0;
    END LOOP;
END;
/

```

Problema 3: DELETE acidental sem WHERE

```
-- Ops! Deletei tudo por engano
DELETE FROM usuario; -- Esqueci WHERE!

-- SOLUÇÃO: ROLLBACK imediatamente
ROLLBACK;

-- Verificar se restaurou
SELECT COUNT(*) FROM usuario;
```

Resumo e Melhores Práticas

Checklist de Segurança

Antes de executar UPDATE/DELETE:

- ☐ Testei com SELECT para ver o que será afetado?
- ☐ Contei quantos registros serão modificados/deletados?
- ☐ Verifiquei dependências e impacto em outras tabelas?
- ☐ Tenho backup dos dados?
- ☐ Estou em ambiente correto (não produção se for teste)?
- ☐ Revisei a cláusula WHERE cuidadosamente?
- ☐ Entendo as consequências da operação?

Durante a execução:

- ☐ Executei em uma transação?
- ☐ Verifiquei o resultado antes de COMMIT?
- ☐ Documentei a operação se for importante?

Após a execução:

- ☐ Confirmei com SELECT que o resultado está correto?
- ☐ Fiz COMMIT ou ROLLBACK apropriadamente?
- ☐ Registre a operação em log/auditoria se necessário?

Comparação UPDATE vs DELETE

Aspecto	UPDATE	DELETE
O que faz	Modifica valores existentes	Remove registros completos
Estrutura	Mantém registros	Remove registros
Reversível	Sim (com ROLLBACK)	Sim (com ROLLBACK antes COMMIT)
Usa WHERE	Recomendado	Obrigatório na prática
Performance	Média	Pode ser lento em grandes volumes

Aspecto	UPDATE	DELETE
Risco	Médio-Alto	Alto
Integridade	Pode violar constraints	Afeta FKs (CASCADE, RESTRICT)

Ordem de Importância - Comandos DML

1. **SELECT** - Leitura (seguro, não destrutivo)
2. **INSERT** - Adiciona dados (baixo risco)
3. **UPDATE** - Modifica dados (médio-alto risco)
4. **DELETE** - Remove dados (alto risco)

Quando Usar Cada Comando

Use UPDATE quando:

- Precisa corrigir dados incorretos
- Dados mudaram e precisam refletir novo estado
- Quer modificar valores mas manter o registro
- Implementa mudança de status

Use DELETE quando:

- Dados não são mais necessários
- Implementa política de retenção
- Remove dados duplicados ou inválidos
- Cumpre regulamentação (LGPD, GDPR)

Use Soft Delete (UPDATE) em vez de DELETE quando:

- Pode precisar dos dados no futuro
- Quer manter histórico
- Auditoria é importante
- Restauração deve ser fácil

Exercícios Práticos

Os exercícios práticos estão disponíveis no diretório [exercicios/](#) e cobrem:

- UPDATE simples e complexo
- DELETE com diferentes condições
- Operações em massa
- Integridade referencial
- Transações e rollback
- Subconsultas em UPDATE/DELETE
- Casos de uso práticos

Referências

- **Elmasri, R. & Navathe, S.** (2016). *Fundamentals of Database Systems*. 7th Edition. Pearson.
- **Date, C.J.** (2012). *SQL and Relational Theory*. 2nd Edition. O'Reilly Media.
- **Beaulieu, A.** (2020). *Learning SQL*. 3rd Edition. O'Reilly Media.
- **Oracle Corporation.** (2024). *Oracle Database SQL Language Reference*.
- **ISO/IEC 9075.** *SQL Standard Documentation*.

Conclusão

UPDATE e DELETE são comandos poderosos mas perigosos. Dominar estas operações com segurança e eficiência é essencial para qualquer profissional de banco de dados. Sempre priorize:

1. **Segurança:** Teste antes, use WHERE, transações
2. **Integridade:** Respeite constraints e relacionamentos
3. **Performance:** Considere índices e volumes de dados
4. **Auditoria:** Documente operações importantes
5. **Reversibilidade:** Sempre tenha plano B (backup, ROLLBACK)

Lembre-se: É melhor ser paranoico com UPDATE/DELETE do que precisar explicar como perdeu dados importantes! 🛡️